

Основы систем мобильной связи (ПОМС)

Отчет по Расчетно-графической работе

Студент: Любимов К. А.

Группа: ИА-331

1. Цель работы:

Закрепить и структурировать знания, полученные в рамках изучения дисциплины «Основы систем мобильной связи».

2. Задание для выполнения практической работы:

- 1) Введите с клавиатуры ваши имя и фамилию латиницей.
- 2) Сформируйте битовую последовательность, состоящую из L битов, кодирующих ваши имя и фамилию латинице ASCII-символов. Результат: массив нулей и единиц с данными и разработанный ASCII-кодер. Визуализируйте последовательность на графике.
- 3) Вычислите CRC длиной M бит для данной последовательности, используя входные данные для своего варианта из работы №5 и добавьте к битовой последовательности. Результат: CRC-генератор и выведенный в терминал CRC.
- 4) Для того, чтобы приемник смог корректно принимать такой сигнал и находить моменты начала, нужно реализовать синхронизацию. Для этого перед отправкой полученной последовательности добавьте последовательность Голда, которую вы реализовывали в работе №4, длиной G -бит. Результат: функция генерации последовательности Голда и массив с битами данных, CRC и синхронизации. Визуализируйте последовательность на графике.
- 5) Преобразуйте биты с данными во временные отсчеты сигналов, так чтобы на каждый бит приходилось N -отсчетов. Результат: массив длиной $N \times (L+M+G)$ нулей и единиц – но это уже временные отсчеты сигнала (пример амплитудной модуляции). Визуализируйте последовательность на графике.
- 6) Создайте нулевой массив длиной $2 \times N \times (L+M+G)$. Введите с клавиатуры число от 0 до $N \times (L+M+G)$ и в соответствие с введенным значением вставьте в него массив значений из п.5. Результат – массив Signal – визуализируйте на графике.
- 7) Предположим, что сформированная выше последовательность, промодулировала высокочастотное несущее колебание, передалась через радиоканал и на приемной стороне была оцифрована с заданной частотой дискретизации f_s (число отсчетов сигнала в 1 секунде). Проходя через канал отсчеты сигнала исказились (опустим пока историю с затуханием и изменением амплитуды) – к ним добавились значения шумов, присутствовавших в канале, которые можно получить, используя нормальный закон распределения с $\mu=0$ и σ – вводится с клавиатуры (float). То есть нужно сформировать массив с шумом размером $2 \times N \times (L+M+G)$, реализовав его с помощью нормального распределения,

например, затем нужно поэлементно сложить информационный сигнал с полученным шумом. Визуализировать массив отсчетов зашумленного принятого сигнала.

8) Реализуйте функцию корреляционного приема и определите, начиная с какого отсчета (семпла) начинается синхросигнал в полученном массиве, удалите лишние биты до этого массива, выведите значение в терминал. Результат: функция корреляционного приемника.

9) Зная длительность в отсчетах N каждого символа, разберите оставшиеся символы. Накапливайте по N отсчетов и сравнивайте их с пороговым значением P (подумайте, какое значение порога следует выбрать, чтобы интерпретировать полученные семплы нулями или единицами). Напишите функцию, которая будет принимать решение по каждому N отсчетам – 0 передавался или 1, на выходе которой должно быть $(L+M+G)$ битов данных. Лишние отсчеты можно отбросить.

10) Удалите из полученного массива G -бит последовательности синхронизации.

11) Проверьте корректность приема бит, посчитав CRC. Выведите в терминал информацию о факте наличия или отсутствия ошибки.

12) Если ошибок в данных нет, то удалит биты CRC и оставшиеся данные подайте на ASCII-декодер, чтобы восстановить посимвольно текст. Выведите результат на экран.

13) Визуализируйте спектр передаваемого и принимаемого (зашумленного) сигналов. Измените длительность символа, уменьшите ее в два раза и увеличьте тоже вдвое. Выведите на одном графике спектры всех трех сигналов (с короткими, средними и длинными символами).

14) Сделайте промежуточные выводы по каждому пункту работы и общее заключение.

15) Оформите работу. Отчет должен содержать титульный лист, содержание, цель и задачи работы, теоретические сведения, исходные данные, этапы выполнения работы, сопровождаемые скриншотами и графиками, демонстрирующими успешность выполнения, и промежуточными выводами, результирующими таблицами и заключение и **ссылка в виде QR-кода на репозиторий с кодом (git).**

Задание на защиту РГР:

1) Построить зависимость вероятности успешного декодирования блока с данными (CRC=OK) от числа семплов на 1 бит (от 1 до 200) и для сигмы = 0,4; 0,6; 0,8; 1,0; 1,8

2) Максимум пика автокорреляции при синхронизации от сигмы (от 0 до 4 сигма) и на этом же графике - вероятность декодирования блоков с данными (несколько кривых - 5/20/100/200 семплов на 1 бит)

3. Выполнения работы:

Я использовал язык C++ и Python для реализации расчетно-графической работы. В качестве справочников я использовал методички всех предыдущих лабораторных работ. Код лежит в репозитории GitHub, ссылка на который приложена в конце отчета. Также я решил воспользоваться своим кодом из предыдущих лабораторных работ, только они были модифицированы мною под РГР.

4. Результаты:

Пример вывода `rgr.cpp`:

```
Enter your name and surname: Kirill Liubimov
```

```
Your fullname in binary:
```

```
0 1 0 0 1 0 1 1
0 1 1 0 1 0 0 1
0 1 1 1 0 0 1 0
0 1 1 0 1 0 0 1
0 1 1 0 1 1 0 0
0 1 1 0 1 1 0 0
0 0 1 0 0 0 0 0
0 1 0 0 1 1 0 0
0 1 1 0 1 0 0 1
0 1 1 1 0 1 0 1
0 1 1 0 0 0 1 0
0 1 1 0 1 0 0 1
0 1 1 0 1 1 0 1
0 1 1 0 1 1 1 1
0 1 1 1 0 1 1 0
```

```
CRC-sequence: 0 1 0 1 0 0 0
```

```
Your fullname in binary + CRC:
```

```
0 1 0 0 1 0 1 1
0 1 1 0 1 0 0 1
0 1 1 1 0 0 1 0
0 1 1 0 1 0 0 1
0 1 1 0 1 1 0 0
0 1 1 0 1 1 0 0
0 0 1 0 0 0 0 0
0 1 0 0 1 1 0 0
0 1 1 0 1 0 0 1
0 1 1 1 0 1 0 1
0 1 1 0 0 0 1 0
0 1 1 0 1 0 0 1
0 1 1 0 1 1 0 1
0 1 1 0 1 1 1 1
0 1 1 1 0 1 1 0
```

```
0 1 0 1 0 0 0
```

Последовательность ЯВЛЯЕТСЯ М-последовательностью

Последовательность Голда:

```
1 1 1 0 0 0 1 1 0 1 1 1 0 1 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 1 1
```

Gold + Data + CRC:

```
1 1 1 0 0 0 1 1
```

```
0 1 1 1 0 1 0 1
```

```
0 0 0 0 1 0 0 1
```

```
0 1 1 0 0 1 1 0
```

```
1 0 0 1 0 1 1 0
```

```
1 1 0 1 0 0 1 0
```

```
1 1 1 0 0 1 0 0
```

```
1 1 0 1 0 0 1 0
```

```
1 1 0 1 1 0 0 0
```

```
1 1 0 1 1 0 0 0
```

```
0 1 0 0 0 0 0 0
```

```
1 0 0 1 1 0 0 0
```

```
1 1 0 1 0 0 1 0
```

```
1 1 1 0 1 0 1 0
```

```
1 1 0 0 0 1 0 0
```

```
1 1 0 1 0 0 1 0
```

```
1 1 0 1 1 0 1 0
```

```
1 1 0 1 1 1 1 0
```

```
1 1 1 0 1 1 0 0
```

```
1 0 1 0 0 0
```

Enter position to insert the signal (0..3160): 100

Enter standard deviation of noise sigma: 0.2

No Errors in data!

Decoded text: Kirill Liubimov

Так выглядит работа программы `rgr.cpp` при этом создаются дополнительные файлы с данными полученными в ходе вычисления

Все созданные файлы сохраняются в отдельную папку `data`:

```
someotb@someotb-B550-GAMING-X-V2:~/Code/OSMS$ ls data -R
```

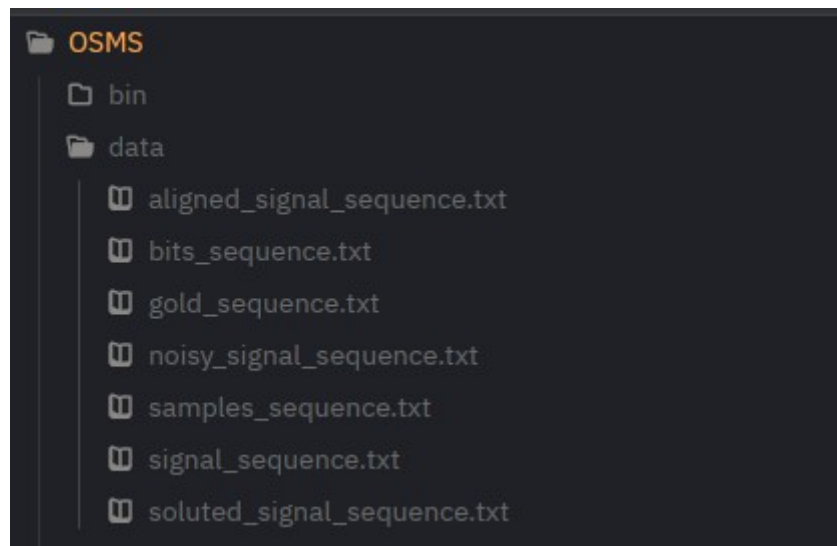
```
data:
```

```
aligned_signal_sequence.txt gold_sequence.txt      samples_sequence.txt
```

```
soluted_signal_sequence.txt
```

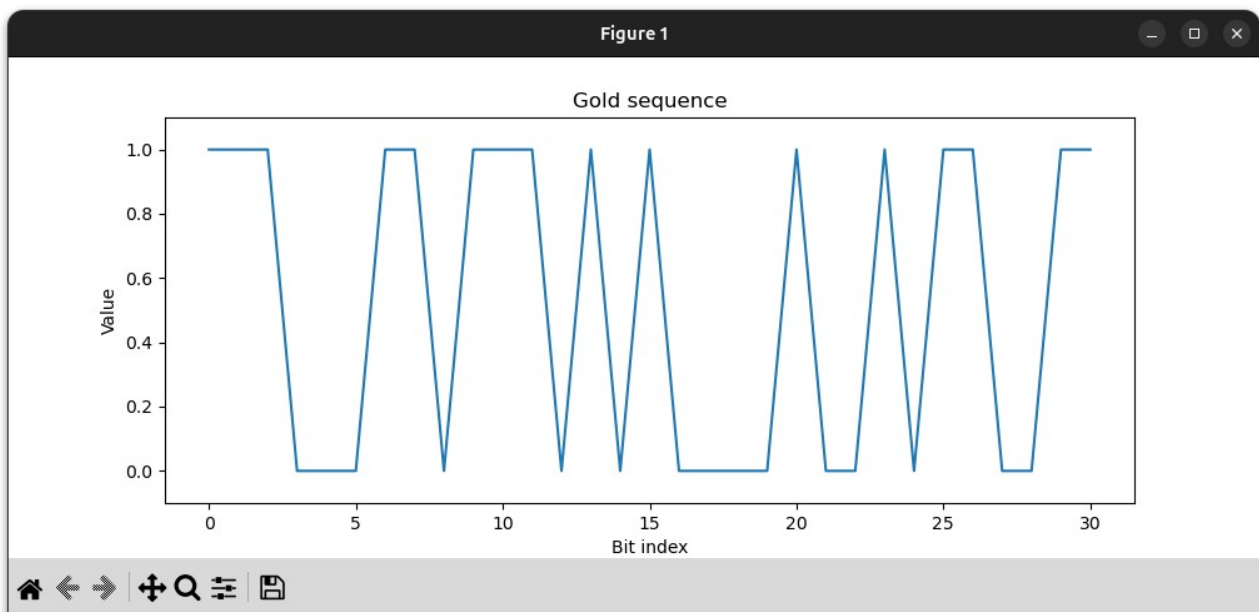
```
bits_sequence.txt      noisy_signal_sequence.txt signal_sequence.txt
```

```
someotb@someotb-B550-GAMING-X-V2:~/Code/OSMS$
```

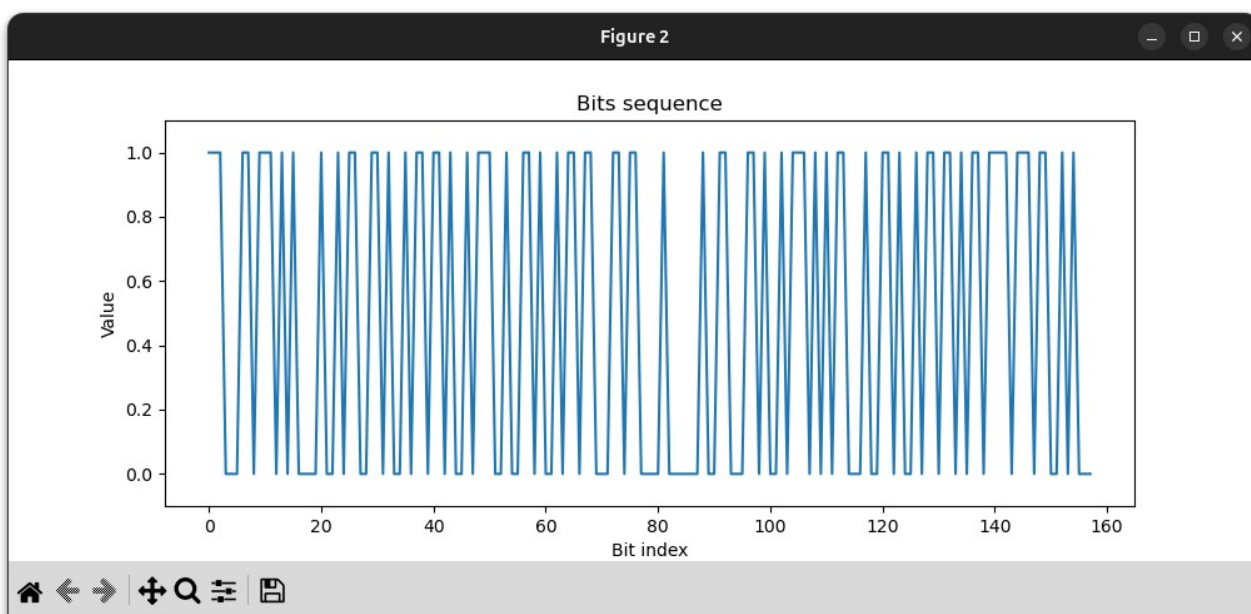


Затем с помощью файла ``rgr.py`` мы можем визуализировать все полученные данные.

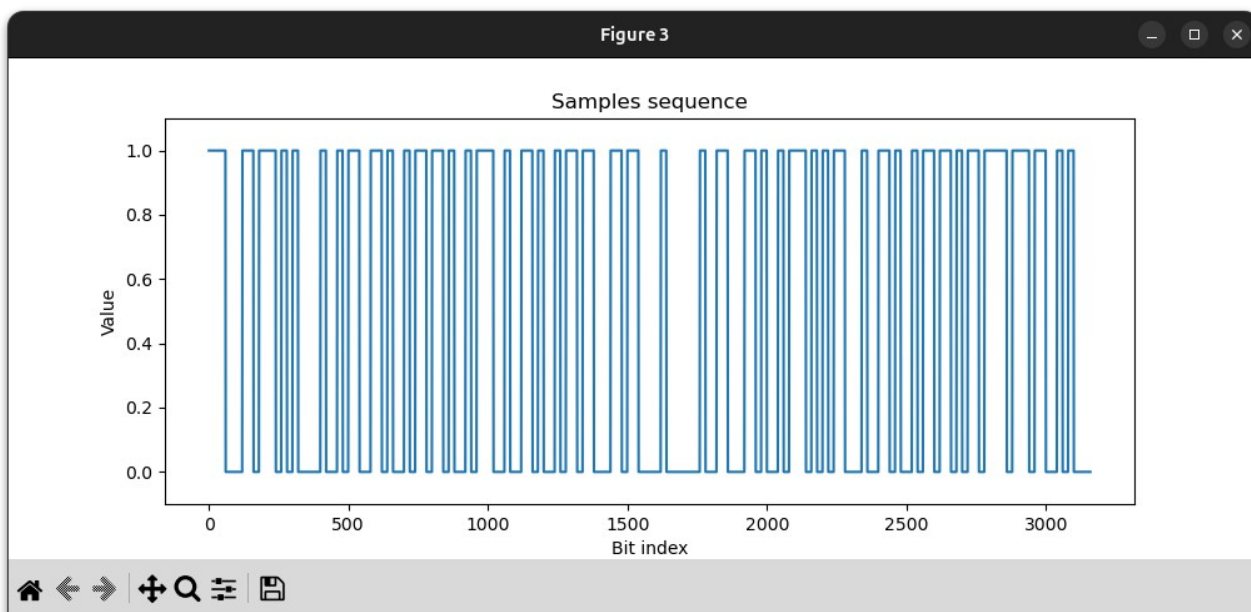
Первый график выводит на экран последовательность голда:



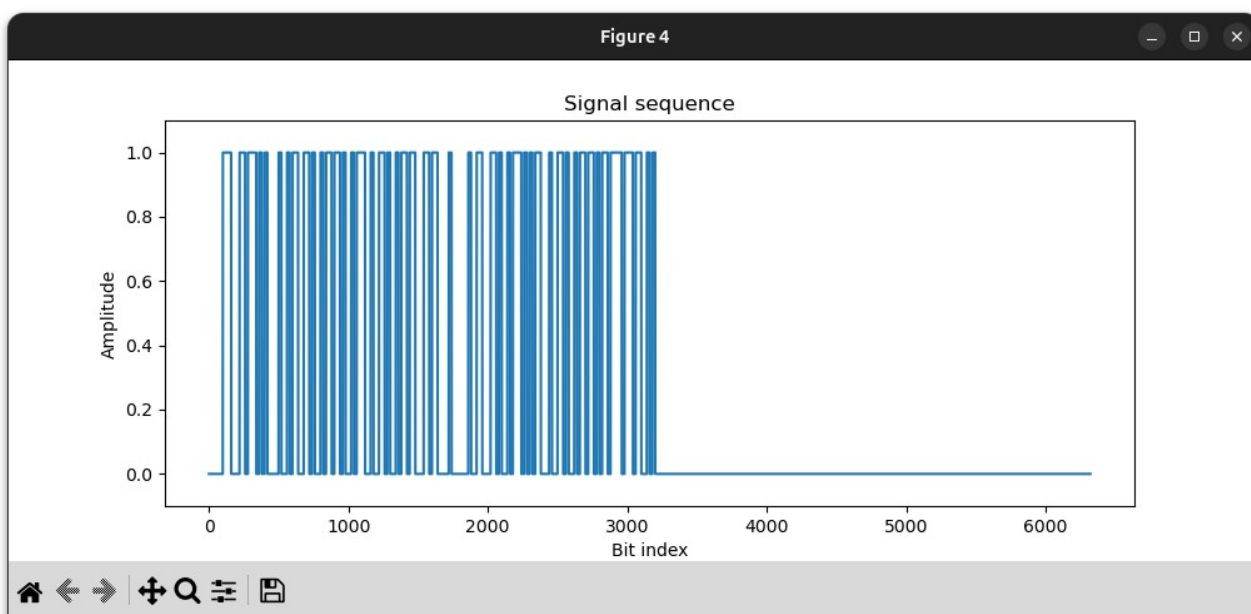
Второй график выводит наше закодированное имя и фамилию + последовательность Голда + CRC, выглядит это так:



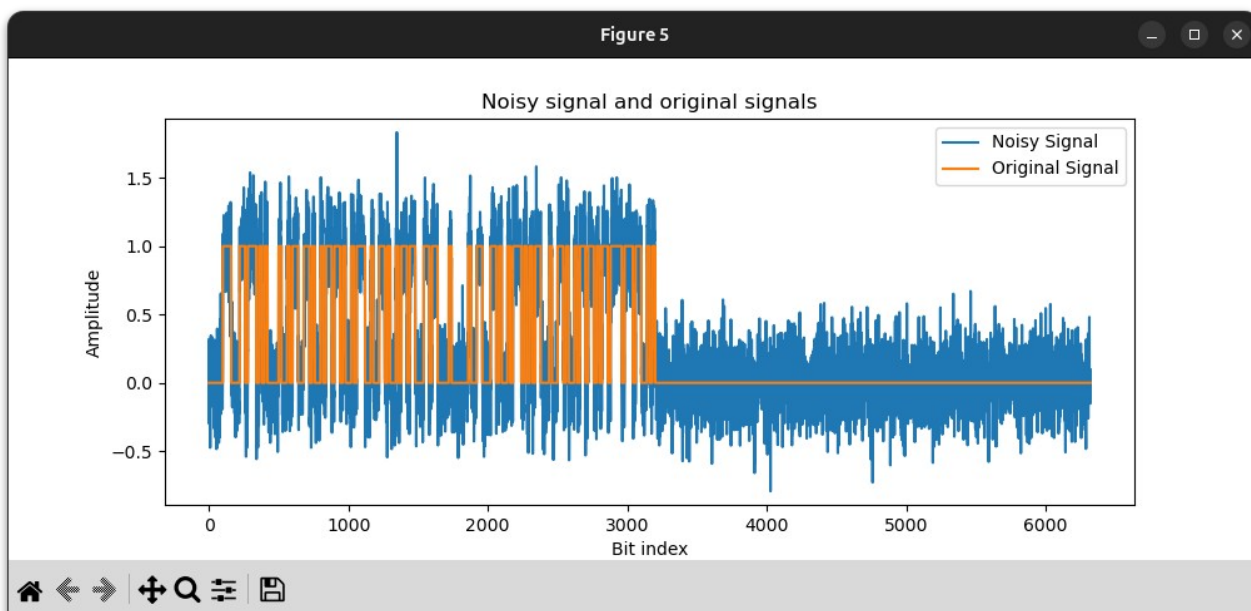
Третий график выводит эту же битовую последовательность только уже в виде сэмплов, на графике видно во сколько раз увеличился каждый бит(мы сделали `upsample`, в моем случае это был $N = 20$):



Четвертый график показывает как длина отправленного сообщения увеличилась в 2 раза, при этом во время выполнения программы мы выбираем позицию с которой начнется непосредственно наш сигнал, это можно отследить в листинге кода, который я приложил в начале этого пункта, там мы выбрали позицию = 100, то есть в нашем отправленном сообщении сигнал начнется с 100 индекса, вот график:

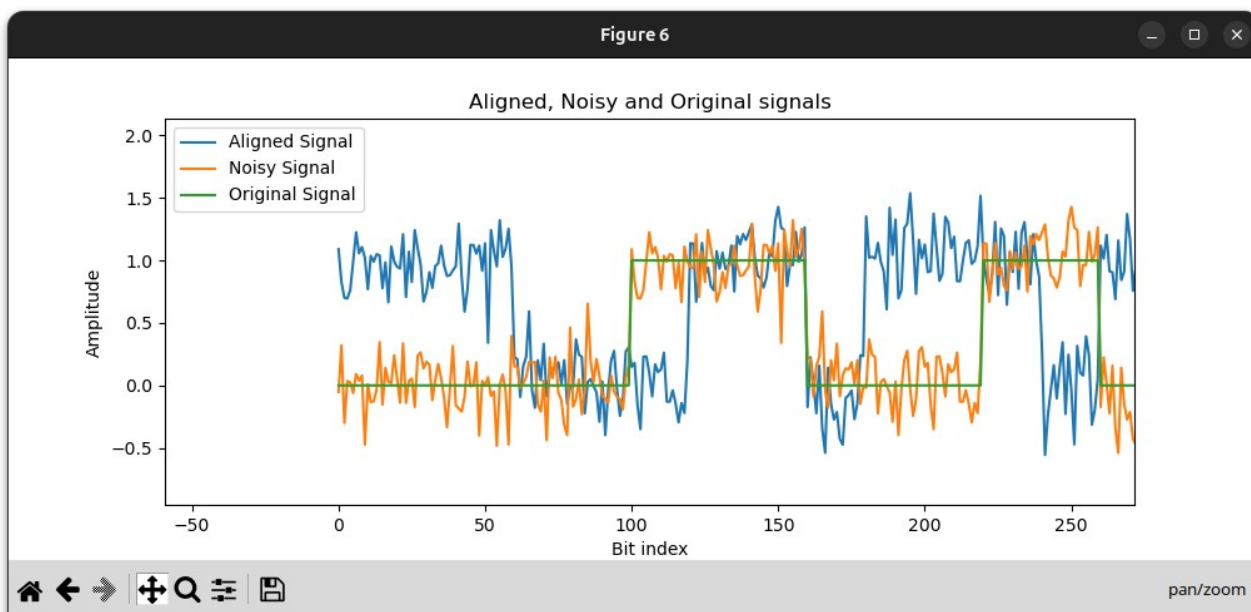
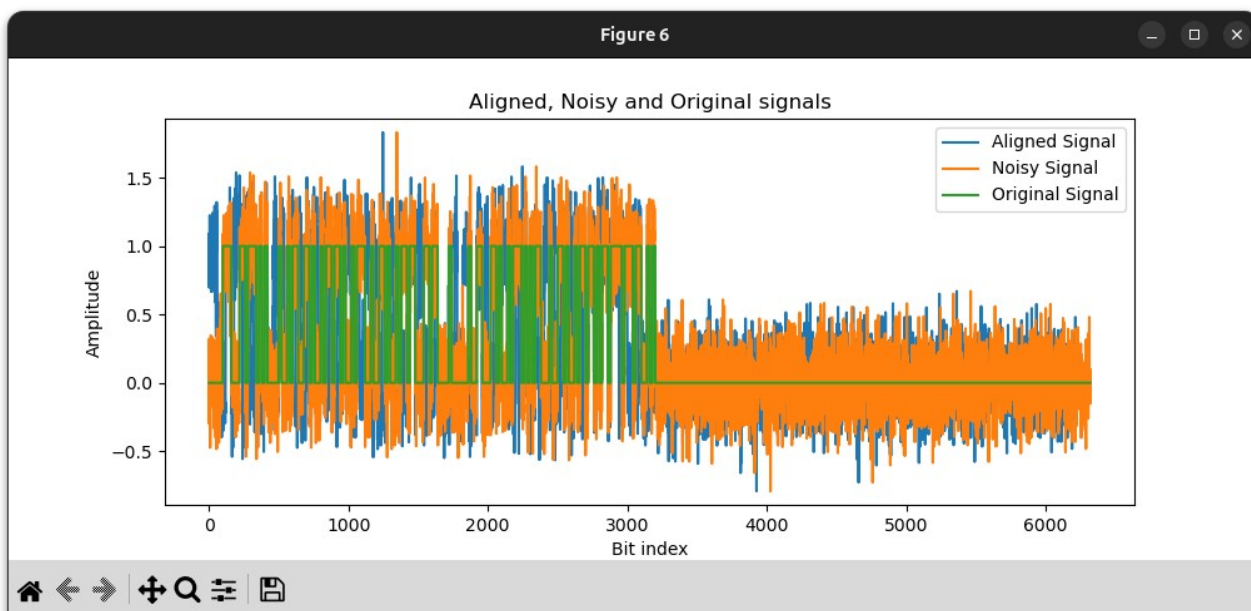


На пятом графике, можно увидеть разницу между зашумленным сигналом, и исходным. Сигнал зашумляется для симуляции реального прохождения сигнала через среду, чтобы проверить как работают методы приема сигнала. По заданию требовалось испортить сигнал, с помощью нормального распределения с кэфициентом σ , который вводится пользователем с клавиатуры, в моем случае при выполнении программы я ввел $\sigma = 0.2$, то есть радиоканал довольно чистый. Вот график:



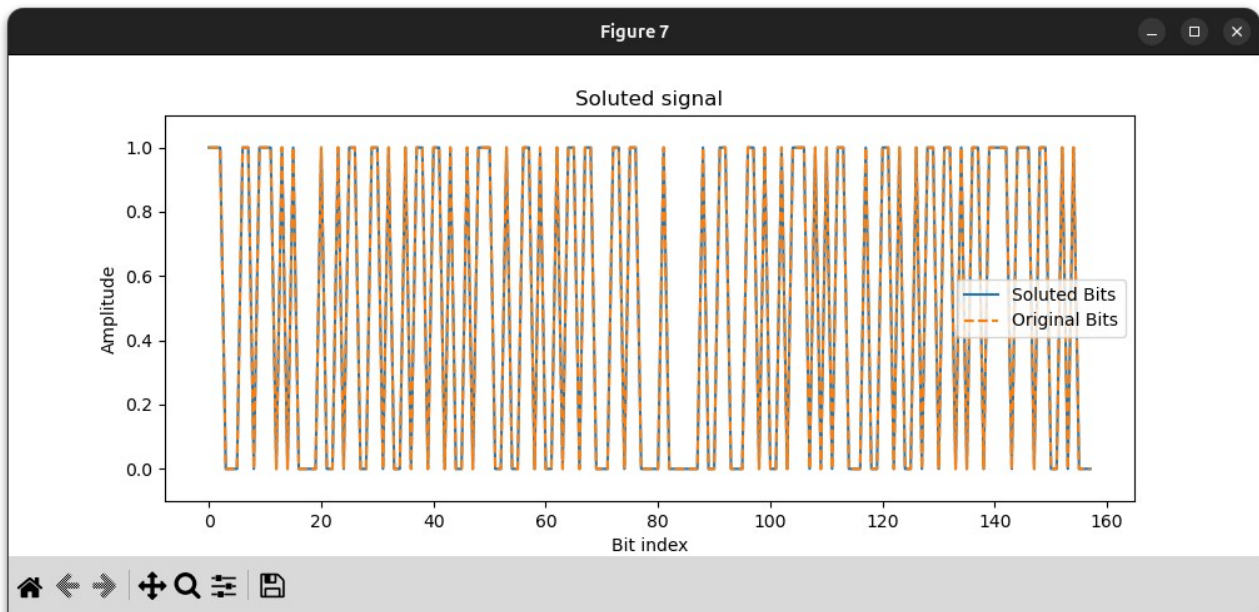
Зашумление просходит путем суммирования исходного значения сигнала с шумом генерируемым программно для каждого значения нашего массива.

На шестом графике я сравниваю 3 массива, исходный, зашумленный и зашумленный сдвинутый, т.к я выбрал не очень удобное значения сдвига = 100, то приложу уточняющий скриншот, на котором видно, что массив сдвинут к 0, то есть в начало сообщения. Надо понимать, что это уже работа приемника, то есть этим будет заниматься принимающее устройство, вот график:

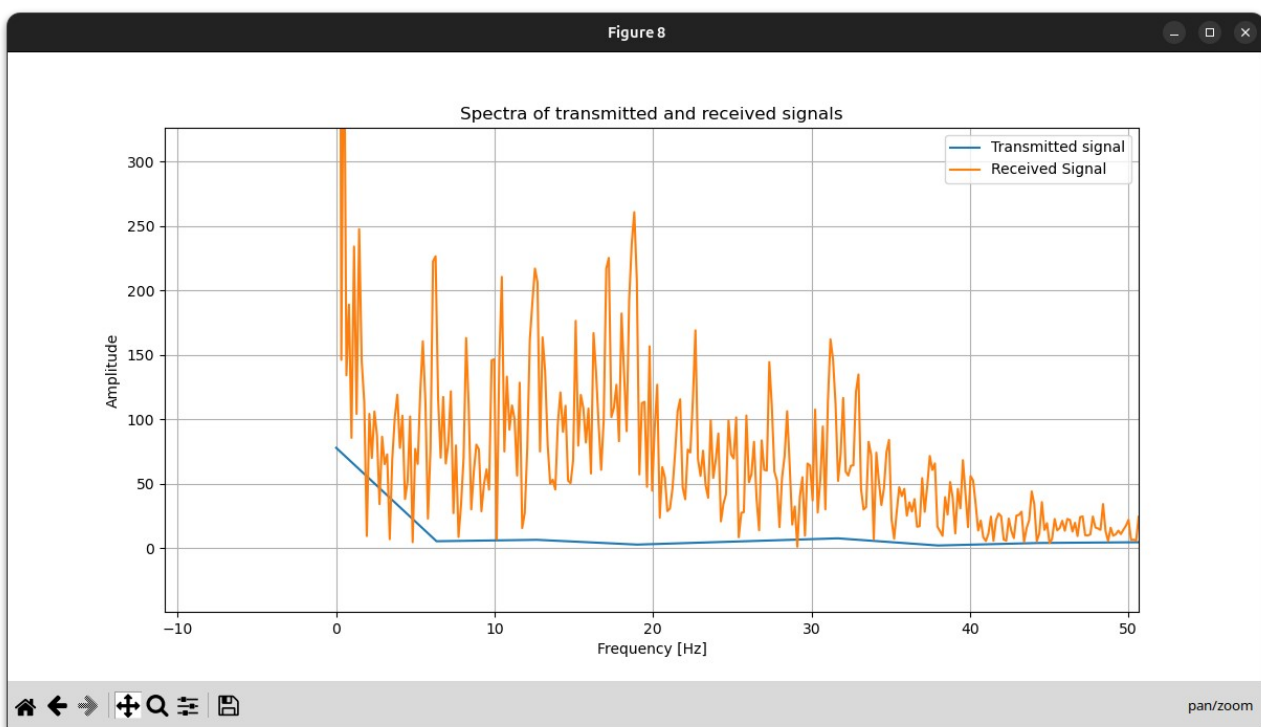
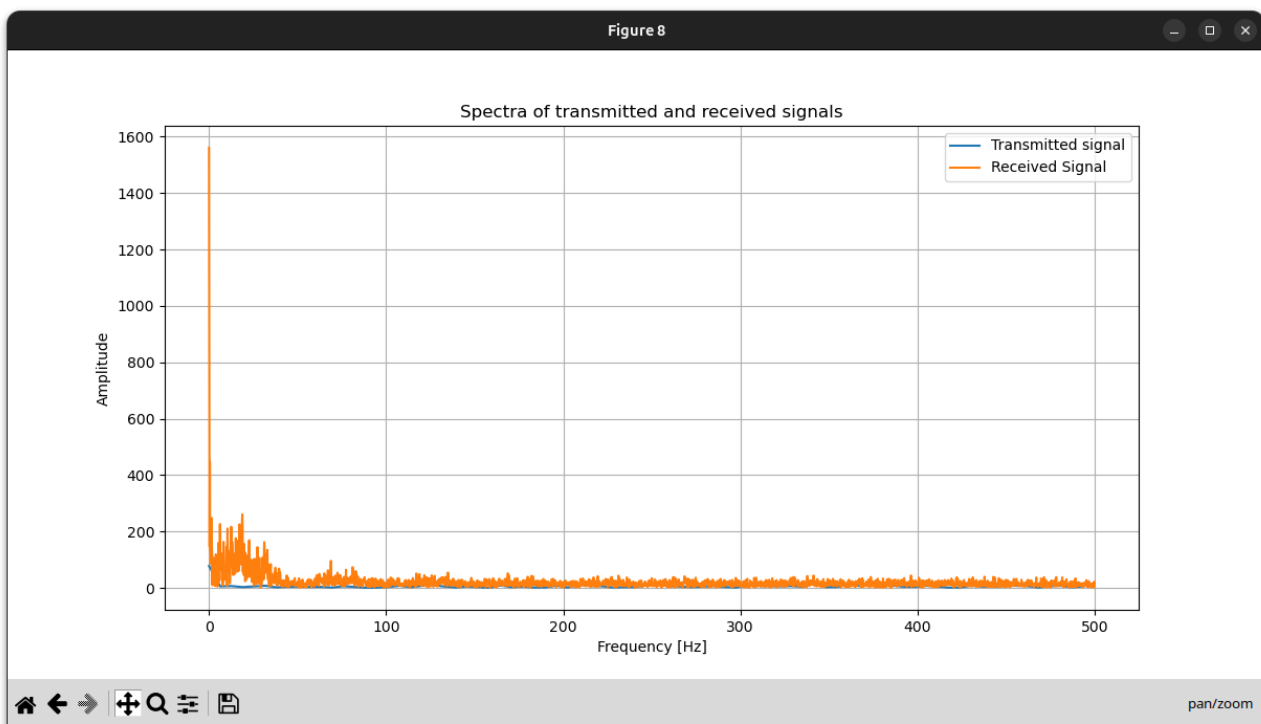


Видно что Aligned Signal сдвинут к 0, делается это для облегчения процесса декодирования.

На седьмом графике видно две линии, одна из которых показывает исходные биты, которые кодируют наше сообщение, а вторая линия показывает, какие биты были выбрано(0 или 1) в ходе декодирования на приемной стороне, в нашем случае, канал был чистым($\sigma = 0.2$) так что для приемной стороны не составило труда выбрать правильные биты, и на графике видно, что все биты декодированной последовательности сходятся и исходной последовательности. График:



Восьмой график показывает спектры исходной битовой последовательности и зашумленной, на графике явно видно, что спектр зашумленного сигнала имеет больше частот в себе, это нормально и очевидно, потому что при генерации шума, мы получаем случайные числа, которые суммируются с нашим сигналом и получается, что сигнала начинает хранить в себе больше разных частот, отсюда и такой график:



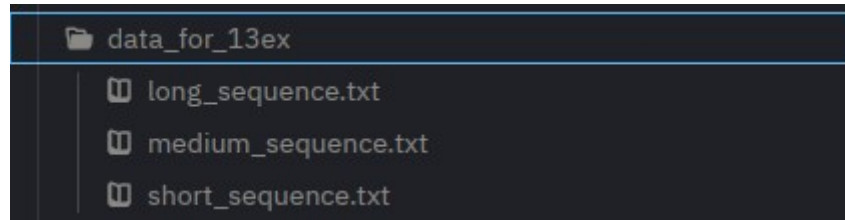
На девятом графике показан частотный спектр для 3 разных сигналов:

Short – это сигнал с $\text{upsample koef} = 5$, то есть на один бит приходится 5 сэмплов

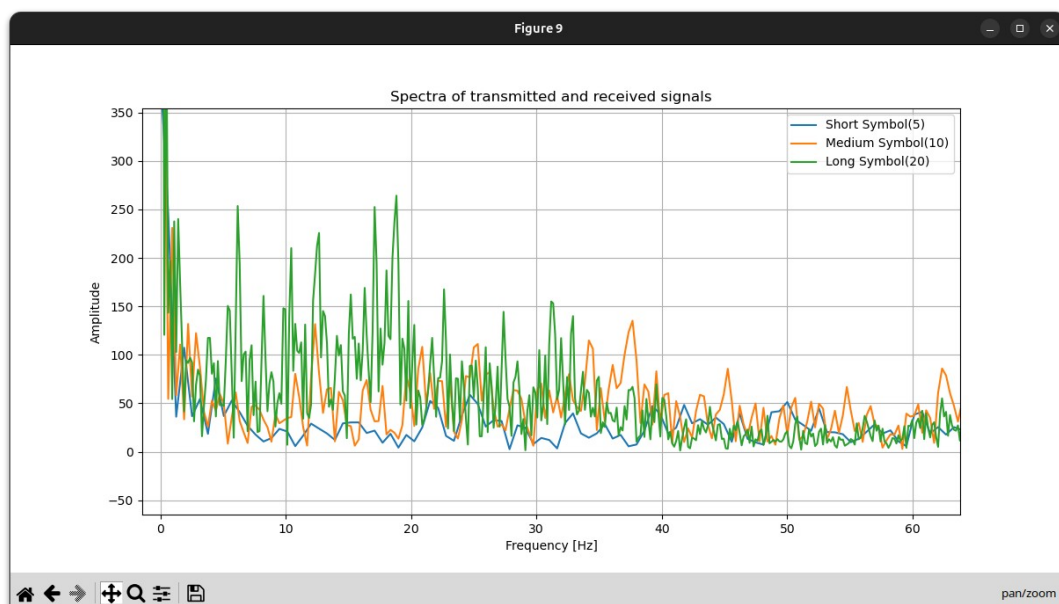
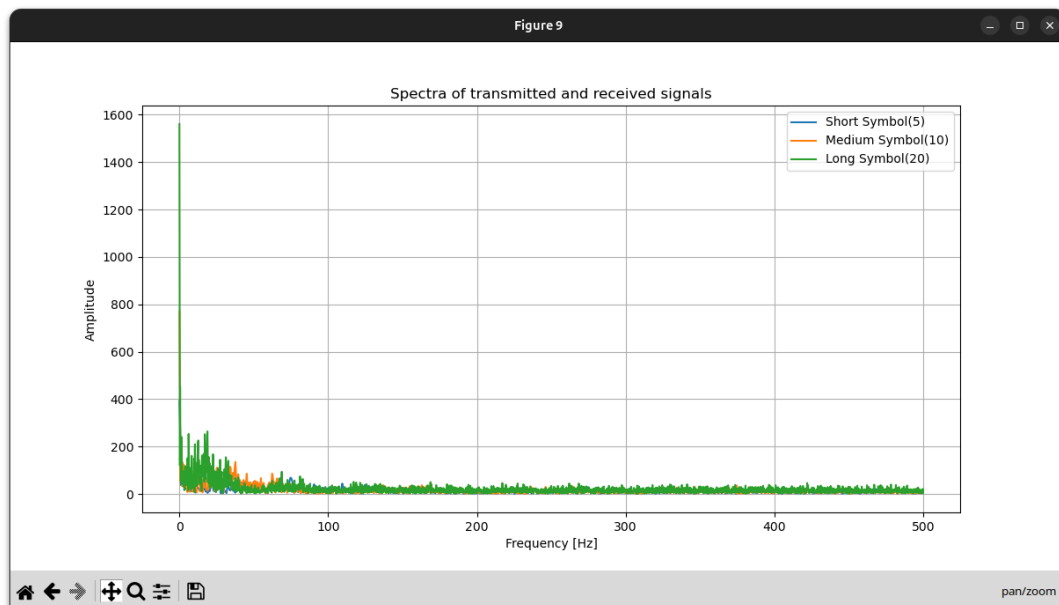
Medium – это сигнал с $\text{upsample koef} = 10$, то есть на один бит приходится 10 сэмплов

Long – это сигнал с $\text{upsample koef} = 20$, то есть на один бит приходится 20 СЭМПЛОВ

Для этого задания я создал специальную папку для хранения данных



Я получил эти файлы 3 раза запуская программу, меняя только $\text{upsample koef}\{5, 10, 20\}$, вводимые пользователем данные были одинаковы, и получился такой график:



Видно что длинные символы дают более узкий и «чистый» спектр, это хорошо для демодуляции и фильтрации, узкие пики легче ловить matched filter'ом.

Короткие символы не такие выраженные в частотной области, меньше устойчивость к шуму, сложнее точно определить корреляцию.

Upsample помогает сделать спектр более точным. Энергия сигналов одинаковая, но распределение по частотам различается. Это напрямую влияет на результаты корреляции и вероятность CRC OK.

Задание на защиту РГР:

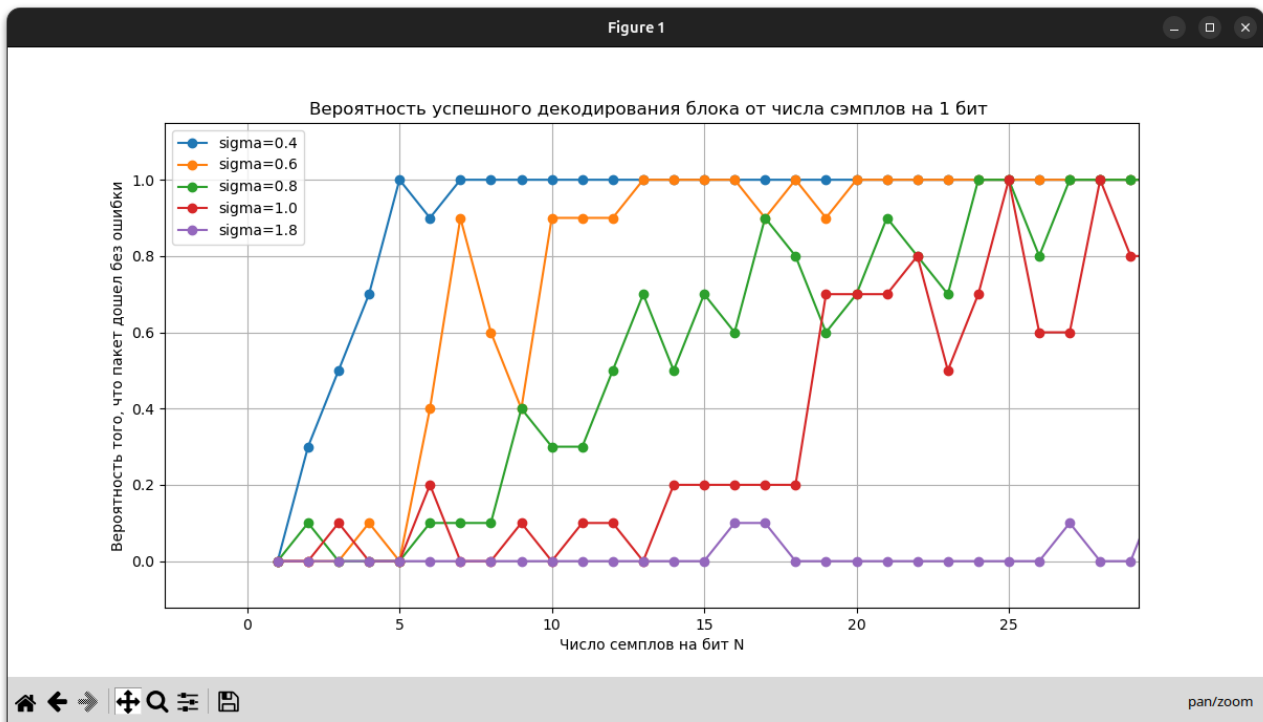
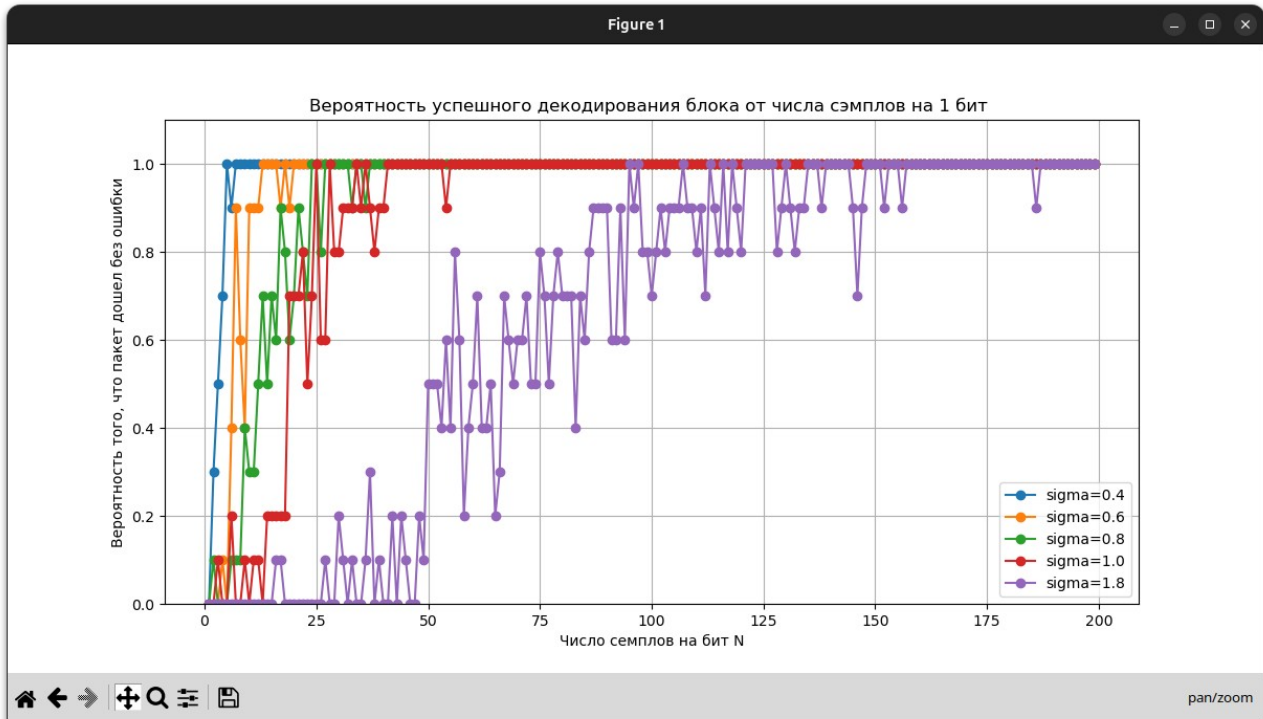
Пример вывода `addit\_task.cpp`:

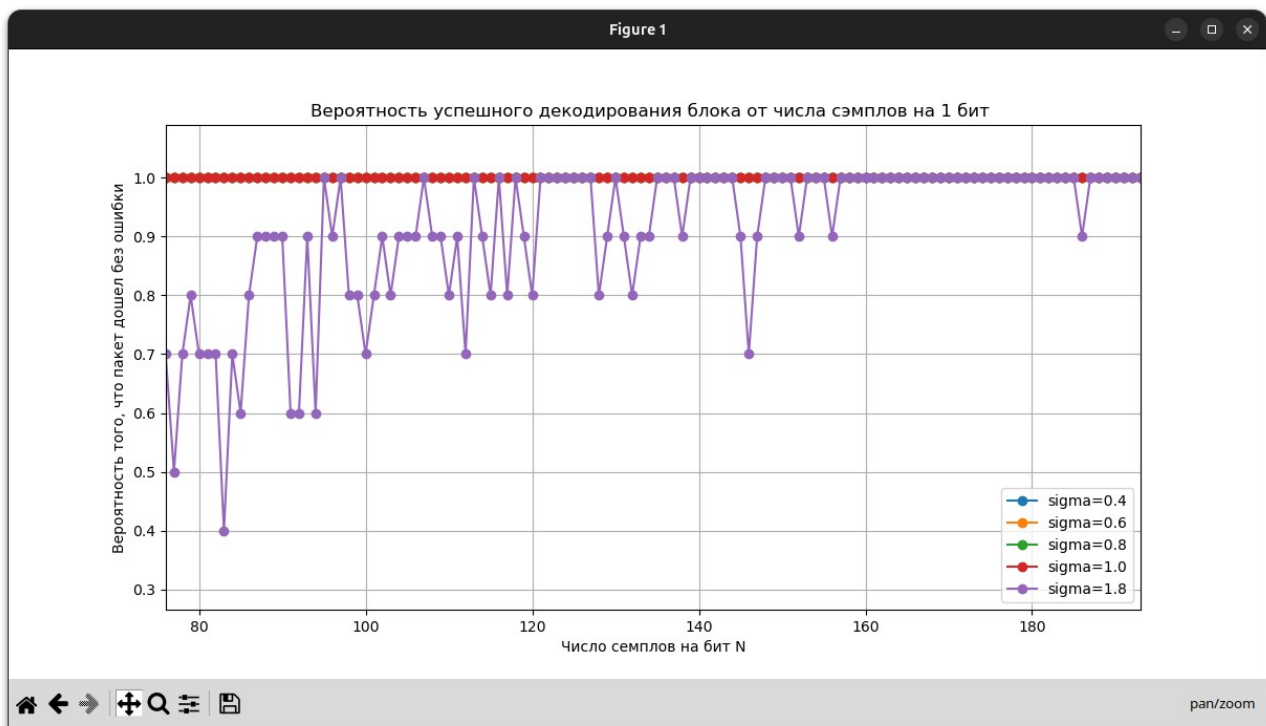
```
Which experiment you wanna try: #1 - (1), #2 - (2): 1
Enter your name and surname: Kirill
Starting experiment #1...
Experiment #1 with: Sigma=0.4, N=1 is running...
Experiment #1 complited with: Sigma=0.4, N=1
Experiment #1 with: Sigma=0.4, N=2 is running...
Experiment #1 complited with: Sigma=0.4, N=2
Experiment #1 with: Sigma=0.4, N=3 is running...
Experiment #1 complited with: Sigma=0.4, N=3
Experiment #1 with: Sigma=0.4, N=4 is running...
...
Experiment #1 complited with: Sigma=1.8, N=195
Experiment #1 with: Sigma=1.8, N=196 is running...
Experiment #1 complited with: Sigma=1.8, N=196
Experiment #1 with: Sigma=1.8, N=197 is running...
Experiment #1 complited with: Sigma=1.8, N=197
Experiment #1 with: Sigma=1.8, N=198 is running...
Experiment #1 complited with: Sigma=1.8, N=198
Experiment #1 with: Sigma=1.8, N=199 is running...
Experiment #1 complited with: Sigma=1.8, N=199
Elapsed time: 3752.12 seconds
```

Хочу сказать, что, я запускаю эксперимент с одними и теми же данными 10 раз, и беру из этих 10 эксперимент среднее значение, так результаты получаются более точными, но это увеличивает время выполнения программы.

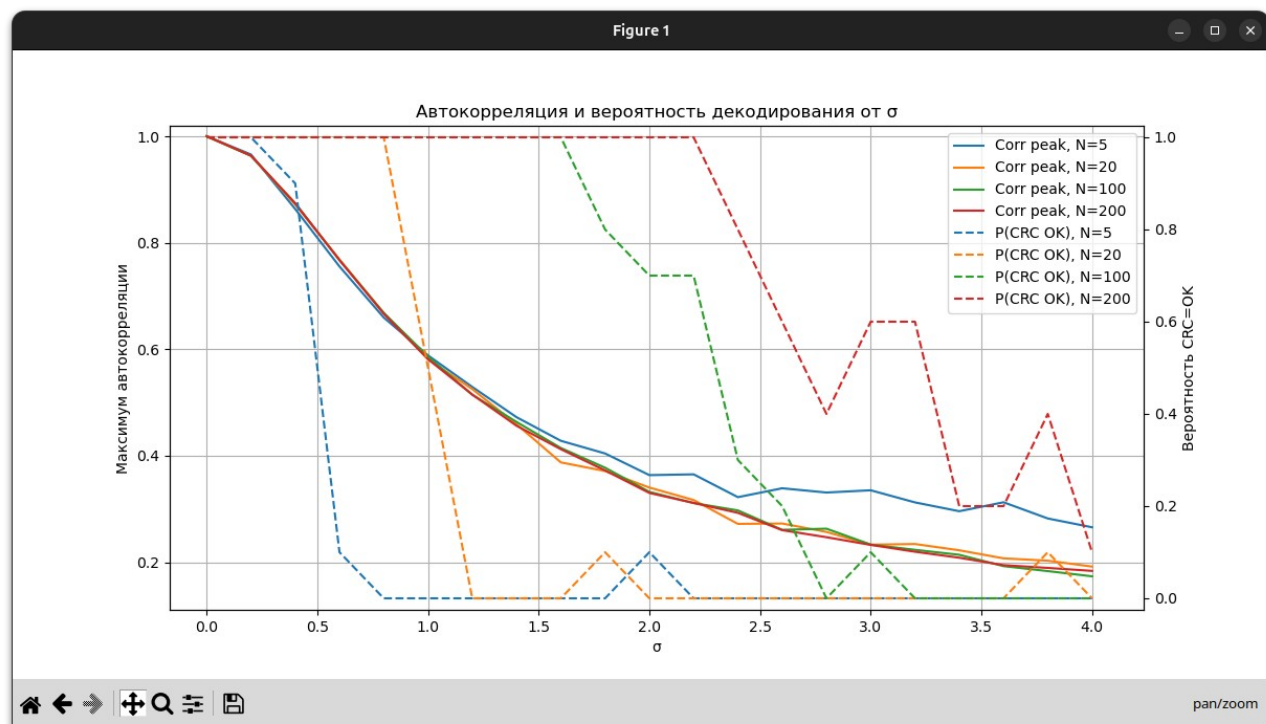
На первом графике видно, что при маленьком кол-во сэмплов на бит, только сообщения которое проходили через чистый радиоканал имеют хорошую вероятности для декодирования, а для $\sigma = 1.8$ стабильное

декодирования происходит только начиная с примерно 160 сэмплов на бит, и то видно что даже после этого значения N могут происходить осечки в декодировании, этот график наглядно показывает, как сильно влияет N на корректный прием сигнала. Вот график:

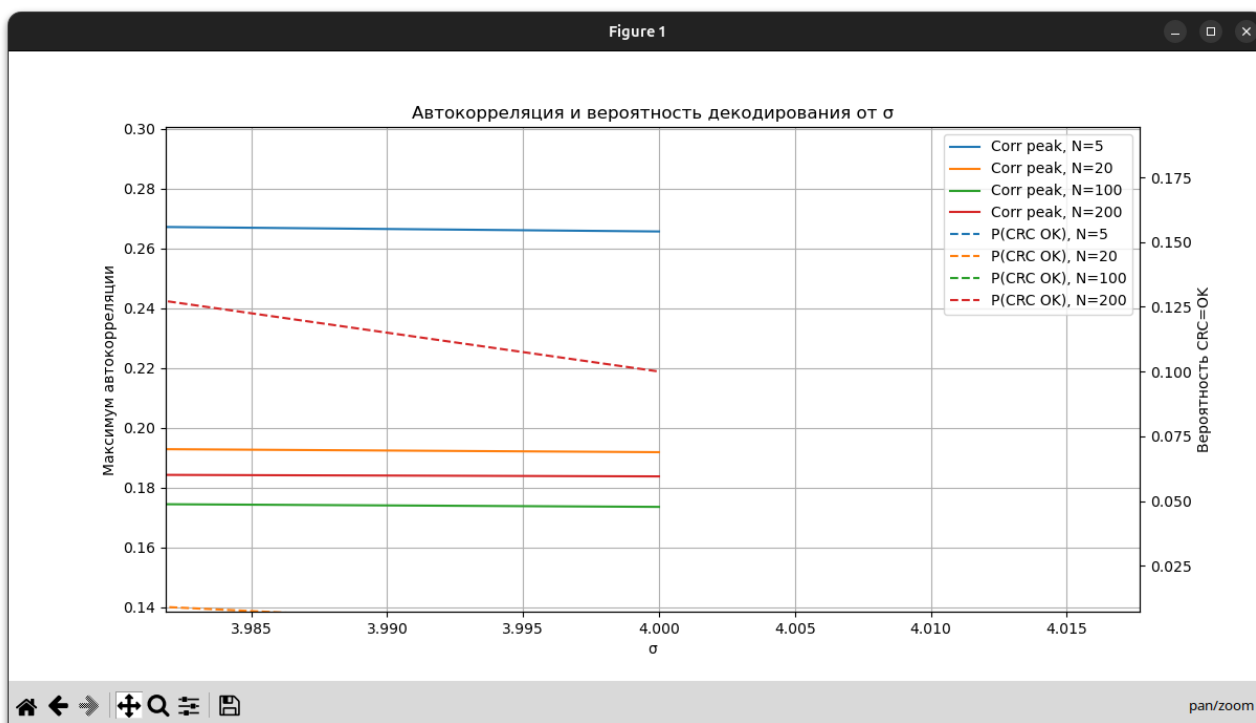




На втором графике показано как зависит значения автокорреляции и CRC ОК от “чистоты” радиоканала. Видно, что при больших значениях σ вероятность декодирования сигнала почти 0, значит что передавая таким способом сообщения мы вряд-ли получим правильные значения на приемной стороне, если будет настолько сильный шум у радиоканала.

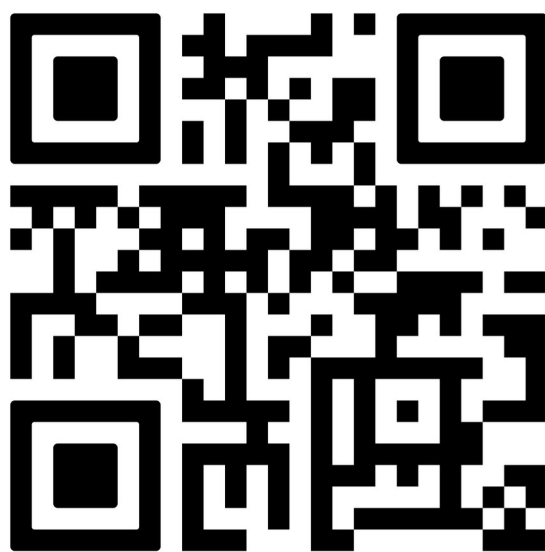


Видно, что сообщения с $N = 5$ не может быть корректно принято уже с $\sigma = 0.75$, но корреляция у этого сигнала выше чем у сигналов с большим N , это значит, что мы хорошо находим начало сигнала, но после неправильно его декодируем, отсюда и низкая вероятность CRC ОК при “грязном” радио канале. Когда N больше, мы сильнее размазываем сигнал, и нам легче понять 1 была закодирована или 0, но тяжелее найти начало сообщения. Видно что при $N = 200$, максимальная зашумленность радиоканала при которой мы можем успешно отправлять и принимать радиосигналы это $\sigma = 2.2$. Далее вероятность успешного декодирования падает и при $\sigma = 4.0$ уже равна 0.100, приближенный график:



5. Вывод:

Я изучил методы кодирования и декодирования сообщений, проходящих через радиоканал, зашумления которого происходило программно. Получил данные, построил зависимости, сделал выводы о том, что кол-во сэмплов на бит, зашумленность радиоканала могут сильно повлиять на процесс передачи. Изучил методы синхронизации и проверки ошибок. Получил наглядные результаты для каждого из пунктов задания и в целом, хорошо провер время.



Ссылка на [GitHub](#)